

Curso: IF0004 - Desarrollo de Software II

Docente: Angélica Ulate Céspedes

Porcentaje: 20%

Modalidad: grupos de 5

Proyecto: App de una veterinaria

Una veterinaria requiere una aplicación web para que sus clientes puedan gestionar la información de sus mascotas y consultar datos relevantes de atención. El sistema debe ofrecer una experiencia clara, rápida y responsiva, enfocada únicamente en el rol cliente.

Objetivo general

Desarrollar una aplicación web en React que permita a un cliente visualizar y administrar información básica de sus mascotas mediante un conjunto mínimo de pantallas y navegación, aplicando buenas prácticas de desarrollo, control de versiones y pruebas E2E.

Tareas por realizar

- Gestión de proyectos en Jira
- Pruebas de integración E2E
- Alcance funcional
- Manejo de versionamiento

Gestión del proyecto en Jira

Este se solicitará en dos partes, la primera debe ser entregada el día 3 de febrero, para ello la docente asignará citas virtuales para la demostración por parte de cada grupo, y la segunda parte se solicitará en la defensa del proyecto. Todos los estudiantes deben conectarse a la llamada.

I Parte: antes de iniciar el desarrollo, el equipo debe:

1. Crear un proyecto en Jira Software.
2. Definir un Backlog.
3. Crear las historias de usuario, respetando el formato (el como, el quiero, el para). Además cada una debe llevar sus criterios de aceptación.
4. Como mínimo deben de existir 7 historias de usuario.
5. Estimar y organizar el trabajo (Story Points o estimación simple).
6. Asignación de las historias a los desarrolladores.

II Parte: durante el proceso, el equipo debe:

1. Crear subtareas por historia.
2. Mover historias por el flujo mínimo: To Do → In Progress → QA → Done, según se vayan avanzando.
3. Registrar comentarios de QA en las historias de usuario durante el proceso de verificación.
4. Adjuntar capturas de pantalla (screenshots) en las historias de usuario como evidencia del trabajo realizado por QA y del avance del proyecto.

Pruebas E2E

El equipo debe:

1. Implementar pruebas E2E usando las herramientas aprendidas.
2. Documentar su ejecución mediante capturas de pantalla asociadas a las historias de usuario en Jira, videos o demás.
3. Verificar que las pruebas de integración se ejecuten tanto durante el desarrollo como nuevamente en la defensa del proyecto en vivo.
4. Se deben implementar como mínimo 6 pruebas E2E distintas (es decir una por pantalla, incluyendo header y footer), cada una validando un flujo completo de la aplicación.

Alcance funcional (pantallas obligatorias)

La aplicación debe incluir, como mínimo, las siguientes vistas:

1. Home
 - Mostrar bienvenida al cliente (nombre).
 - Mostrar un resumen general (ejemplo: cantidad de mascotas registradas, próxima cita o recordatorio simulado).
 - Proveer accesos rápidos hacia “Mis Mascotas” y “Perfil del Cliente”.
2. Pet List
 - Presentar un listado de mascotas del cliente en tarjetas o tabla.
 - Cada mascota debe mostrar: nombre, especie, raza (opcional), edad (opcional) y estado (opcional).
 - Debe permitir acceder al Perfil de la Mascota al seleccionar una mascota.
3. Pet Profile
 - Mostrar información detallada de una mascota seleccionada.
 - Debe incluir datos mínimos: nombre, especie, raza, edad, peso (opcional), alergias (opcional).



- Debe incluir secciones o pestañas (tabs) o bloques: “Resumen”, “Vacunas” y “Consultas” (los datos pueden ser simulados).

4. Client Profile

- Mostrar y permitir editar información del cliente: nombre, correo, teléfono.
- Incluir botón de guardar cambios (persistencia local o simulada).

5. HEADER (componente obligatorio)

- Logo o nombre de la veterinaria (clickeable → Home)
- Menú de navegación
- (Opcional) botón de Cerrar Sesión
- Debe Mostrarse en todas las páginas
- Mantenerse fijo o consistente al navegar

6. FOOTER (componente obligatorio)

- Texto de derechos:
- “© 2026 Veterinaria Web. Todos los derechos reservados.”
- Páginas sociales como facebook...

La información puede ser hardcodeada o simulada mediante archivos locales (JSON o LocalStorage). No se requiere backend ni consumo de APIs. El código debe ser escrito en el lenguaje de inglés.

Manejo de versionamiento

1. Deben manejar un branch principal
2. Crear diferentes ramas.
3. Debe realizar pull request, por lo que los compañeros deben revisar el código, dar sugerencias en caso de ser necesario y autorizar el mismo.
4. En caso de haber conflictos solucionarse.
5. Deben compartir el repositorio con la docente



Condiciones de entrega

- El proyecto será realizado en grupos.
- La entrega se hará únicamente por mediación virtual (un archivo comprimido por grupo), en el [README.md](#) puede adjuntar los links y otro tipo de documentación que necesite.
- La fecha y hora límite son estrictas: no se aceptan entregas posteriores.
- A partir de 1 semana antes de la fecha de entrega no se atenderán consultas de ningún tipo.
- Cada estudiante es responsable de que el grupo mantenga en todo momento una versión que compile y ejecute sin errores.
- La participación se evaluará con base en evidencias objetivas: asignaciones en Jira, commits en Git, participación en la defensa y respuestas a preguntas técnicas. Por lo que cada estudiante debe hacerse responsable de que su trabajo realizado sea visible.

Defensa del proyecto

- Cada grupo debe realizar una defensa oral con la profesora, la cual se realizará por citas presenciales, **el día 25 de febrero**. En caso de no poder asistir el estudiante pierde el porcentaje de la misma.
- En la defensa deben:
 - Mostrar en vivo la ejecución de la aplicación.
 - Mostrar las pruebas E2E.
 - Mostrar el git y el Jira
- Todos los integrantes deben participar y demostrar conocimiento del proyecto completo.
- La calificación de la defensa es grupal. El desempeño individual impacta directamente la nota final del grupo.



Rúbricas:

Defensa Oral (8%)

1. Demostración en vivo

- Ejecución correcta de la aplicación.
- Las funcionalidades principales solicitadas se muestran en tiempo real.
- No se presentan errores que impidan la demostración de la aplicación.

2. Preguntas específicas del docente

- Cada estudiante responde únicamente cuando la docente se lo solicita.
- El grupo presenta de forma ordenada las ideas, respetando los tiempos indicados.
- Respuestas claras, seguras, acertadas y técnicas.
- Se evalúa que cada estudiante responda lo que se le solicita en relación con el proyecto.

3. Dominio global del proyecto

- Todos los integrantes deben demostrar que conocen no solo “su parte” sino también cómo encaja en el proyecto completo.
- Se observa coherencia entre respuestas de diferentes integrantes.

4. Trabajo en equipo y comunicación

- Interacción respetuosa, colaboración en la defensa, sin interrupciones ni contradicciones.

Código Fuente (5%)

- El proyecto está desarrollado utilizando React, según lo solicitado.
- Se utilizan las herramientas requeridas en el enunciado del proyecto.
- La aplicación incluye todas las visitas obligatorias solicitadas.
- La navegación entre pantallas funciona correctamente.
- La aplicación presenta funcionalidad completa según el alcance definido.
- El código fuente corresponde a lo demostrado durante la defensa.

Backlog de Jira (3 %)

- El proyecto está creado en Jira Software.
- Existe un backlog definido dentro del proyecto.
- El backlog contiene al menos 7 historias de usuario.
- Todas las historias de usuario están redactadas en el formato: Como [rol], quiero [acción], para [beneficio].
- Las historias están escritas desde el punto de vista del usuario y no desde una perspectiva técnica.
- Todas las historias de usuario incluyen criterios de aceptación Y los criterios de aceptación son claros, verificables y coherentes con la historia.
- Cada historia de usuario permite determinar claramente cuándo se considera terminada.

Todas las historias de usuario cuentan con una estimación asignada y se utiliza un único método de estimación (Story Points o estimación simple) de manera consistente.

- Las estimaciones asignadas son coherentes con la complejidad del trabajo.



- Todas las historias de usuario están asignadas a un desarrollador responsable.
- La distribución de historias entre los desarrolladores es razonable.
- Todas las historias de usuario incluyen subtareas.
- Las subtareas describen actividades técnicas claras y relacionadas con la historia.
- El tablero del proyecto incluye las columnas: To Do, In Progress, QA y Done.
- Las historias de usuario se mueven correctamente a través del flujo definido.
- La columna QA se utiliza antes de marcar una historia como Done.
- El estado Done solo se utiliza cuando la historia cumple todos los criterios de aceptación.
- Se presenta evidencia visual (capturas) del backlog y del tablero en Jira.
- El trabajo en Jira se realiza antes de iniciar el desarrollo del proyecto.

E2E (3%)

- El proyecto incluye pruebas E2E utilizando una herramienta válida (Cypress o Katalon).
- Las pruebas E2E cubren flujos completos de la aplicación.
- Las pruebas E2E se ejecutan correctamente durante la defensa en vivo.
- Se evidencia que las pruebas E2E fueron ejecutadas durante el desarrollo del proyecto.
- Se adjuntan capturas de pantalla de las pruebas E2E dentro de las historias de usuario en Jira.
- Las pruebas E2E corresponden a la aplicación que se defiende.



Control de versiones (1%):

- El proyecto utiliza un sistema de control de versiones (Git).
- El repositorio contiene commits realizados durante el desarrollo del proyecto.
- Los commits reflejan avances reales del proyecto.
- El código presentado en la defensa corresponde al repositorio entregado.